

Αλγοριθμική Επιστήμη Δεδομένων Δεύτερη Σειρά Γραπτών Ασκήσεων

Παναγή Αντρέας
Α.Μ. : 7115 1424 00008

Π.Μ.Σ. - ΑΛΜΑ

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εθνικό Μετσόβιο Πολυτεχνείο
Αθήνα, Ελλάδα *

Ιούνιος 2024



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

* Στοιχειοθέτηση με χρήση $\text{\LaTeX}$

Περιεχόμενα

Άσκηση 1	1
Εκφώνηση	1
Άσκηση 8.2.1	1
Άσκηση 8.4.1	1
Λύση	1
Άσκηση 8.2.1	1
Άσκηση 8.4.1	3
Άσκηση 2	4
Εκφώνηση	4
Άσκηση 9.2.3	4
Άσκηση 9.3.2	4
Λύση	4
Άσκηση 9.2.3.	4
Άσκηση 9.3.2	5
Άσκηση 3	7
Εκφώνηση	7
Άσκηση 5.2.2	7
Άσκηση 5.2.3	7
Λύση	7
Άσκηση 5.2.2	7
Άσκηση 5.2.3	7
Άσκηση 4	9
Εκφώνηση	9
Λύση	9
Άσκηση 5	12
Εκφώνηση	12
Λύση	12

Άσκηση 1

Εκφώνηση

Άσκηση 8.2.1

A popular example of the design of an on-line algorithm to minimize the competitive ratio is the ski-buying problem. Suppose you can buy skis for \$100, or you can rent skis for \$10 per day. You decide to take up skiing, but you don't know if you will like it. You may try skiing for any number of days and then give it up. The merit of an algorithm is the cost per day of skis, and we must try to minimize this cost

One on-line algorithm for making the rent/buy decision is “buy skis immediately.” If you try skiing once, fall down and give it up, then this on-line algorithm costs you \$100 per day, while the optimum off-line algorithm would have you rent skis for \$10 for the one day you used them. Thus, the competitive ratio of the algorithm “buy skis immediately” is at most $1/10$ th, and that is in fact the exact competitive ratio, since using the skis one day is the worst possible outcome for this algorithm. On the other hand, the on-line algorithm “always rent skis” has an arbitrarily small competitive ratio. If you turn out to really like skiing and go regularly, then after n days, you will have paid \$10n or \$10/day, while the optimum off-line algorithm would have bought skis at once, and paid only \$100, or \$100/n per day.

Your question: design an on-line algorithm for the ski-buying problem that has the best possible competitive ratio. What is that competitive ratio?

Hint: Since you could, at any time, have a fall and decide to give up skiing, the only thing the on-line algorithm can use in making its decision is how many times previously you have gone skiing.

Άσκηση 8.4.1

Using the simplifying assumptions of Example 8.7, suppose that there are three advertisers, A , B , and C . There are three queries, x , y , and z . Each advertiser has a budget of 2. Advertiser A bids only on x ; B bids on x and y , while C bids on x , y , and z . Note that on the query sequence $xyyzzz$, the optimum off-line algorithm would yield a revenue of 6, since all queries can be assigned.

1. Show that the greedy algorithm will assign at least 4 of these 6 queries.
2. Find another sequence of queries such that the greedy algorithm can assign as few as half the queries that the optimum off-line algorithm assigns on that sequence.

Λύση

Άσκηση 8.2.1

Έστω:

- B : το κόστος αγοράς σκι.
- R : το κόστος ενοικίασης σκι για μία ημέρα.
- d : ο άγνωστος συνολικός αριθμός ημερών που το άτομο θα κάνει σκι.

Ο βέλτιστος off-line αλγόριθμος (που γνωρίζει το d εκ των προτέρων) θα:

- Ενοικίασε σκι για d ημέρες αν $d \cdot R < B$.
Το συνολικό κόστος θα ήταν $d \cdot R$.
- Αγόρασε σκι αμέσως αν $B \leq d \cdot R$.
Το συνολικό κόστος θα ήταν B .

Συνοπτικά, το βέλτιστο off-line κόστος είναι $\min(d \cdot R, B)$.

Βέλτιστος on-line Αλγόριθμος

1. Ενοικίασε σκι καθημερινά για τις πρώτες $k - 1$ ημέρες, όπου $k = \lfloor B/R \rfloor$.
2. Εάν το σκι συνεχιστεί για την k -οστή ημέρα, αγόρασε σκι την k -οστή ημέρα.

Λόγος Προσέγγισης Αλγορίθμου

Ο λόγος προσέγγισης του αλγορίθμου είναι $2 - R/B$. Απόδειξη

- Περίπτωση 1: Το σκι σταματά την ημέρα $d < B/R$.
 - Κόστος on-line αλγορίθμου: $d \cdot R$ (ενοικίασε για d ημέρες).
 - Κόστος βέλτιστου αλγορίθμου: $d \cdot R$ (θα είχε επίσης ενοικιάσει).
 - Λόγος: 1.
- Περίπτωση 2: Το σκι σταματά την ημέρα $d = B/R$.
 - Κόστος on-line αλγορίθμου: $(B/R - 1) \cdot R + B = B - R + B = 2B - R$ (ενοικίασε για $B/R - 1$ ημέρες, μετά αγόρασε την ημέρα B/R).
 - Κόστος βέλτιστου αλγορίθμου: B (θα είχε αγοράσει από την αρχή).
 - Αναλογία: $(2B - R)/B = 2 - R/B$. Αυτό είναι το χειρότερο σενάριο.
- Περίπτωση 3: Το σκι συνεχίζεται για $d > B/R$ ημέρες.
 - Κόστος διαδικτυακού αλγορίθμου: $(B/R - 1) \cdot R + B = 2B - R$ (ενοικίασε για $B/R - 1$ ημέρες, μετά αγόρασε).
 - Κόστος βέλτιστου μη-διαδικτυακού: B (θα είχε αγοράσει από την αρχή).
 - Αναλογία: $(2B - R)/B = 2 - R/B$. Η αναλογία δεν ξεπερνά το $2 - R/B$.

Συνεπώς ο μέγιστος λόγος είναι $2 - R/B$.

Εφαρμογή με νούμερα της άσκησης

Δεδομένα:

- Κόστος αγοράς σκι (B) = \$100
- Κόστος ενοικίασης σκι (R) = \$10 ανά ημέρα

Πρώτα, υπολογίζουμε $k = B/R = \$100/\$10 = 10$.

Βέλτιστος on-line Αλγόριθμος για αυτήν την Περίπτωση:

1. Ενοικίασε σκι καθημερινά για τις πρώτες 9 ημέρες.
2. Εάν το σκι συνεχιστεί για τη 10η ημέρα, αγόρασε σκι τη 10η ημέρα.

Λόγος ειδικής περίπτωσης:

Η αναλογία ανταγωνιστικότητας είναι $2 - R/B = 2 - (\$10/\$100) = 2 - 0.1 = 1.9$.

Άσκηση 8.4.1

1. Για να αποφύγουμε την καταμέτρηση όλων των δυνατών περιπτώσεων (οι οποίες προκύπτουν από την τυχαία επιλογή κάπου advertiser με θετικό budget και ενδιαφέρον για την τρέχουσα διαφήμιση) θα δείξουμε ότι στην χειρότερη περίπτωση (σενάριο με θετική πιθανότητα και χαμηλότερο κέρδος) ο αλγόριθμος βρίσκει 4 από τα 6 στην ακολουθία $xyyz$.

Η χειρότερη περίπτωση θα ήταν ο αλγόριθμος αρχικά για τα πρώτα δύο x να επιλέξει τον C ο οποίος αποτελεί τον μόνο που ενδιαφέρεται για όλα τα queries, έπειτα θα συνέχιζε στα y αναγκαστικά μόνο με τον B και τέλος για τα z δεν θα επέλεγε κανέναν, συνεπώς 4 από τα 6

2. Η μια ακολουθία που πληρεί τις απαιτήσεις είναι η εξής : $xxzzzz$.

Ο off-line αλγόριθμος θα επέλεγε τον $AACC$ — — (πρώτες 2 φορές επιλέγει τον A , έπειτα δύο φορές τον C και στα τελευταία δύο queries καμία επιλογή), συνεπώς 4/6.

Ο on-line θα μπορούσε να επιλέξει τον C δύο φορές και έπειτα κανέναν, οπότε 2/6.

$$\text{Άρα } \frac{2/6}{4/6} = \frac{1}{2}$$

Άσκηση 2

Εκφώνηση

Άσκηση 9.2.3

A certain user has rated the three computers as follows:

A: 4 stars, B: 2 stars, C: 5 stars.

The three computers are the following:

Feature	A	B	C
Processor Speed (GHz)	3.06	2.68	2.92
Disk Size (GB)	500	320	640
Main-Memory Size (GB)	6	4	6

- Normalize the ratings for this user.
- Compute a user profile for the user, with components for processor speed, disk size, and main memory size.

Άσκηση 9.3.2

In this exercise, we cluster items in the following matrix. Do the following steps.

	A	B	C	D	E	F	G	H
User1	5	3		2	4		1	
User2		4	2	1		5		
User3	2		5	4		3	2	
User4	4	3	1	2	5			
User5	3	2	4		1		5	

- Cluster the eight items hierarchically into four clusters. The following method should be used to cluster. Replace all 3's, 4's, and 5's by 1 and replace 1's, 2's, and blanks by 0. Use the Jaccard distance to measure the distance between the resulting column vectors. For clusters of more than one element, take the distance between clusters to be the minimum distance between pairs of elements, one from each cluster.
- Then, construct from the original matrix of Fig. 9.8 a new matrix whose rows correspond to users, as before, and whose columns correspond to clusters. Compute the entry for a user and cluster of items by averaging the nonblank entries for that user and all the items in the cluster.
- Compute the cosine distance between each pair of users, according to your matrix from Part (b).

Λύση

Άσκηση 9.2.3.

(α) Κανονικοποίηση των βαθμολογιών του χρήστη:

- Υπολογίζουμε τον μέσο όρο των αξιολογήσεων:

$$\bar{r} = \frac{4 + 2 + 5}{3} = \frac{11}{3} \approx 3,6667.$$

- Αφαιρούμε τον μέσο όρο από κάθε αξιολόγηση:

$$\tilde{r}_A = 4 - \frac{11}{3} = \frac{1}{3} \approx 0,3333, \quad \tilde{r}_B = 2 - \frac{11}{3} = -\frac{5}{3} \approx -1,6667, \quad \tilde{r}_C = 5 - \frac{11}{3} = \frac{4}{3} \approx 1,3333.$$

(6) Υπολογισμός προφίλ του χρήστη ως διάνυσμα με συνιστώσες την ταχύτητα επεξεργαστή, το μέγεθος δίσκου και τη μνήμη.

- Ορίζουμε τα διανύσματα χαρακτηριστικών:

$$\mathbf{f}_A = (3,06, 500, 6), \quad \mathbf{f}_B = (2,68, 320, 4), \quad \mathbf{f}_C = (2,92, 640, 6).$$

- Το προφίλ του χρήστη είναι το σταθμισμένο άθροισμα:

$$\mathbf{p} = \tilde{r}_A \mathbf{f}_A + \tilde{r}_B \mathbf{f}_B + \tilde{r}_C \mathbf{f}_C = \frac{1}{3}\mathbf{f}_A - \frac{5}{3}\mathbf{f}_B + \frac{4}{3}\mathbf{f}_C.$$

- Υπολογίζοντας:

$$\mathbf{p} \approx (0,447, 486,667, 3,333).$$

Άσκηση 9.3.2

(α') Παρακάτω με χρήση κώδικα python εκτελούμε το clustering.

```
import numpy as np
from itertools import combinations

# Original data
data = {
    'a': [4, np.nan, 2],
    'b': [5, 3, np.nan],
    'c': [np.nan, 4, 1],
    'd': [5, 3, 3],
    'e': [1, 1, np.nan],
    'f': [np.nan, 2, 4],
    'g': [3, 1, 5],
    'h': [2, np.nan, 3]
}

# Step 1: Convert to binary matrix according to the rule
def convert_to_binary(vec):
    return [1 if x in [3, 4, 5] else 0 for x in vec]

binary_data = {k: convert_to_binary(v) for k, v in data.items()}

# Step 2: Jaccard distance between binary vectors
def jaccard_distance(u, v):
    intersection = sum(1 for a, b in zip(u, v) if a == b == 1)
    union = sum(1 for a, b in zip(u, v) if a == 1 or b == 1)
    return 1 - intersection / union if union != 0 else 0.0

# Step 3: Clustering setup
clusters = [[k] for k in binary_data]

# Step 4: Agglomerative clustering using min inter-cluster distance
while len(clusters) > 4:
```

```

min_dist = float('inf')
pair_to_merge = None

for i, j in combinations(range(len(clusters)), 2):
    cluster_i = clusters[i]
    cluster_j = clusters[j]

    min_pair_dist = min(
        jaccard_distance(binary_data[a], binary_data[b])
        for a in cluster_i for b in cluster_j
    )

    if min_pair_dist < min_dist:
        min_dist = min_pair_dist
        pair_to_merge = (i, j)

i, j = pair_to_merge
clusters[i].extend(clusters[j])
del clusters[j]

# Final clusters
for idx, cluster in enumerate(clusters, 1):
    print(f"Cluster {idx}: {cluster}")

```

Το αποτέλεσμα είναι:

	a	b	c	d	e	f	g	h
A	1	1	0	1	0	0	1	0
B	0	1	1	1	0	0	0	0
C	0	0	0	1	0	1	1	1

Χρησιμοποιώντας Jaccard distance και ενώνοντας clusters μέχρι να μείνουν 4 τελικά clusters, παρατηρούμε τα εξής αποτελέσματα:

- **Cluster 1:** a, b, d, g
- **Cluster 2:** c
- **Cluster 3:** e
- **Cluster 4:** f, h

(b) Ο πίνακας είναι ο εξής:

	Cluster 1	Cluster 2	Cluster 3	Cluster 4
A	1.0	0.0	0.0	0.0
B	0.75	1.0	0.0	0.0
C	1.0	0.0	0.0	1.0

- (c)
- Cosine απόσταση μεταξύ **A** και **B**: 0.4000
 - Cosine απόσταση μεταξύ **A** και **C**: 0.2929
 - Cosine απόσταση μεταξύ **B** και **C**: 0.5757

Άσκηση 3

Εκφώνηση

Άσκηση 5.2.2

Using the method of Section 5.2.1, represent the transition matrices of the following graphs:

(a) Figure 5.4.

(b) Figure 5.7.

Άσκηση 5.2.3

Using the method of Section 5.2.4, represent the transition matrices of the graph of Fig. 5.3, assuming blocks have side 2.

Λύση

Άσκηση 5.2.2

Παρακάτω φαίνεται ο πίνακας για το Figure 5.4:

Πίνακας 1: Figure 5.4		
Source	Degree	Destinations
A	3	C, D, B
B	2	A, D
C	1	E
D	2	C, B
E	0	—

Παρακάτω φαίνεται ο πίνακας για το Figure 5.7:

Πίνακας 2: Figure 5.7		
Source	Degree	Destinations
a	3	a, b, c
b	2	a, c
c	2	a, b

Άσκηση 5.2.3

Κόμβοι: A, B, C, D; Λωρίδες: $\{A, B\}, \{C, D\}$; $k = 2$

Εξώβαδοι κορυφών στο γράφημα:

$$d_A = 3, \quad d_B = 2, \quad d_C = 0, \quad d_D = 2$$

Χωρίζουμε τον πίνακα σε 4 κομμάτια, επομένως θα επικεντρωθούμε στις σχέσεις των κορυφών μεταξύ των τεσσάρων αυτών “κομματιών” του πίνακα

Μπλοκ (1,1): Γραμμές {A,B}, Στήλες {A,B}

Στήλη j	d_j	Γραμμή i με $M_{ij} \neq 0$
B	2	A

Μπλοκ (1,2): Γραμμή {A,B}, Στήλη {C,D}

Στήλη j d_j Γραμμή i με $M_{ij} \neq 0$

D	2	B
---	---	---

Μπλόκ (2,1): Γραμμή {C,D}, Στήλη {A,B}

Στήλη j d_j Γραμμή i με $M_{ij} \neq 0$

A	3	C, D
---	---	------

B	2	D
---	---	---

Μπλοκ (2,2): Γραμμή {C,D}, Στήλη {C,D}

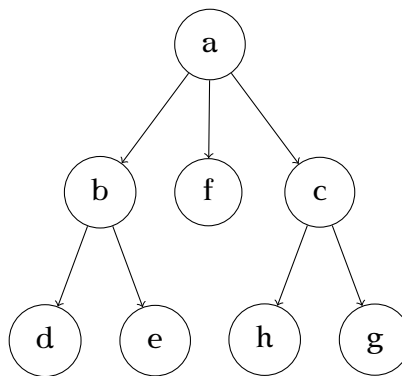
Στήλη j d_j Γραμμή i με $M_{ij} \neq 0$

D	2	C
---	---	---

Άσκηση 4

Εκφώνηση

- (α) Περιγράψτε λεπτομερώς, σε μορφή ψευδοκώδικα, τον αλγόριθμο Girvan-Newman του βιβλίου LRU (κεφ. 10.2.3, 10.2.4) για υπολογισμό της τιμής edge betweenness σε ένα γράφημα. Εξηγήστε την πολυπλοκότητα του αλγορίθμου.
- (β) Εφαρμόστε τον αλγόριθμο Girvan-Newman στον γράφο του σχ. 10.3 του βιβλίου LRU. Αντιστοιχούν τα αποτελέσματα που βρήκατε στις εμπειρικές παρατηρήσεις;
- (γ) Πώς μπορεί να απλοποιηθεί ο αλγόριθμος αν ο γράφος είναι δέντρο; Ποια είναι η πολυπλοκότητα του απλοποιημένου αλγορίθμου;
- (δ) Βρείτε το edge betweenness των ακμών (b, a) , (a, f) και (c, g) του παρακάτω γράφου:



Λύση

- (α) Παρακάτω φαίνεται σε φυσική γλώσσα και ψευδοκώδικα ο αλγόριθμος των Girvan-Newman (για τον υπολογισμό του betweenness)
Σε κάθε επανάληψη του αλγορίθμου υπολογίζεται εκ νέου το betweenness και η ακμή με το μεγαλύτερο αφαιρείται.

Algorithm 1: High Level Girvan–Newman Edge Betweenness

Input: Graph $G = (V, E)$ **Output:** Edge betweenness scores $\text{Betweenness}[e]$ for all $e \in E$

```
1 foreach  $e \in E$  do
2   |  $\text{Betweenness}[e] \leftarrow 0$ 
3 end
4 foreach node  $S \in V$  do
    // Step 1: Run BFS from  $S$  and count shortest paths
5   Perform a breadth first search from  $S$  to discover each node's level.
6   Set  $\text{numPaths}[Y]$  for each node  $Y$  on the first level
7   Calculate  $\text{numPaths}[Y]$  for each other node  $Y$  by summing the  $\text{numPaths}$  of its
    parents in the above level of the BFS tree.

    // Step 2: Compute node credits
8   Assign each leaf node a credit of 1.
9   For each non leaf node  $Z$ , set (one plus the score of the children of the below level)

        
$$\text{nodeCredit}[Z] = 1 + \sum_{\text{edges } (Z \rightarrow W)} \text{edgeCredit}[Z, W]$$


    // Step 3: Compute edge credits and accumulate
10  For each edge  $(Y \rightarrow Z)$  in the BFS DAG, compute

        
$$\text{edgeCredit}[Y, Z] = \text{nodeCredit}[Z] \times \frac{\text{numPaths}[Y]}{\sum_{P \in \text{Parents}[Z]} \text{numPaths}[P]}$$


    Then add this value into  $\text{Betweenness}[Y, Z]$ .
11 end
12 if  $G$  is undirected then
13   foreach  $e \in E$  do
14     |  $\text{Betweenness}[e] \leftarrow \text{Betweenness}[e]/2$ 
15   end
16 end
17 return  $\text{Betweenness}$ 
```

- (6) Εκτελούμε τον αλγόριθμο στο figure 10.3 του βιβλίου και υπολογίζουμε τα παρακάτω αποτελέσματα: Παρατηρούμε ότι πράγματι επιβεβαιώνουν τι εμπειρικές μας παρατηρήσεις η οποίες

Edge	Betweenness
{A,B}	5.0
{A,C}	1.0
{B,C}	5.0
{B,D}	12.0
{D,E}	4.5
{D,F}	4.0
{D,G}	4.5
{E,F}	1.5
{F,G}	1.5

προβλέπουν την ακμή $\{B, D\}$ να αποτελεί διαχωριστή των κοινοτήτων $\{A,B,C\}$ και $\{D,E,F,G\}$

(γ) Έστω $T = (V, E)$ δέντρο με $|V| = n$ κορυφές, το *edge betweenness* μπορεί να υπολογιστεί πιο αποδοτικά

1. Επιλέγουμε έναν κόμβο r ως ρίζα.
2. Με μία εκτέλεση DFS (ή BFS) κάνουμε post-order traversal και υπολογίζουμε για κάθε κόμβο v το $\text{size}[v]$: το πλήθος κόμβων στο υποδέντρο του v .
3. Για κάθε ακμή $e = (u, v)$ όπου u είναι ο γονέας του v , κόβοντας την ακμή αυτή το δέντρο χωρίζεται σε δύο μέρη:
 - το υποδέντρο του v , μέγεθος $s = \text{size}[v]$,
 - το υπόλοιπο δέντρο, μέγεθος $n - s$.

Συνεπώς:

$$\text{betweenness}(e) = s \times (n - s).$$

(δ) Στο συγκεκριμένο παράδειγμα:

$$\text{betweenness}(\{a, b\}) = 3 \cdot 5 = 15$$

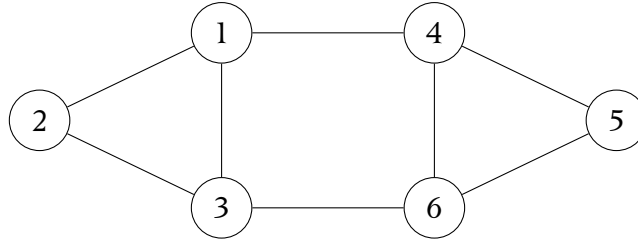
$$\text{betweenness}(\{a, f\}) = 1 \cdot 7 = 7$$

$$\text{betweenness}(\{c, g\}) = 7 \cdot 1 = 7$$

Άσκηση 5

Εκφώνησηση

Για τον γράφο του Σχήματος 10.16 [LRU], υπολογίστε:



- (α) την τιμή της normalized cut, για την διαμέριση $\{1, 2, 3\}, \{4, 5, 6\}$,
 (β) την τιμή modularity για την παραπάνω διαμέριση, καθώς και για τη διαμέριση $\{1\}, \{2, 3\}, \{4, 5\}, \{6\}$.
 (γ) Σχολιάστε τα αποτελέσματα που βρήκατε.

Λύση

- (α) Θεωρούμε την διαμέριση $S := \{1, 2, 3\}, T := \{4, 5, 6\}$.
 Υπολογίζουμε τις τιμές $cut(S, T) = 2$ καθώς και $vol(S) = 5, vol(T) = 5$.
 Συνεπώς

$$Normalized-Cut(S, T) = \frac{2}{5} + \frac{2}{5} = \frac{4}{5}.$$

- (β) Θυμίζουμε ότι ο ορισμός του modularity είναι ο εξής:

$$Q = \frac{1}{2m} \sum_{i,j \in V} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j).$$

Ορίζουμε $A := \{1, 2, 3\}$ και $B := \{4, 5, 6\}$

- Συνολικός αριθμός ακμών: $m = 8$, άρα $2m = 16$.
- Οι βαθμοί των κορυφών είναι οι εξής:

$$k_1 = 3, k_2 = 2, k_3 = 3, k_4 = 3, k_5 = 2, k_6 = 3.$$

- Για το σύνολο A ισχύει ότι:

$$\sum_{i,j \in A} A_{ij} = 2 \cdot 3 = 6, \quad \sum_{i,j \in A} k_i k_j = (3 + 2 + 3)^2 = 8^2 = 64.$$

Άρα

$$\sum_{i,j \in A} \left(A_{ij} - \frac{k_i k_j}{2m} \right) = 6 - \frac{64}{16} = 6 - 4 = 2.$$

- Όσο αφορά το σύνολο B λόγω συμμετρικότητας ισχύει:

$$\sum_{i,j \in B} A_{ij} = 6, \quad \sum_{i,j \in B} k_i k_j = 64,$$

και

$$\sum_{i,j \in B} \left(A_{ij} - \frac{k_i k_j}{2m} \right) = 6 - 4 = 2.$$

Συνεπώς,

$$Q = \frac{1}{2m} (2 + 2) = \frac{4}{16} = 0,25.$$

Τώρα όσο αφορά την διαμέριση $A := \{1\}$, $B := \{2, 3\}$, $C := \{4, 5\}$, $D := \{6\}$
Για κάθε κοινότητα X υπολογίζουμε

$$S_X = \sum_{i,j \in X} \left(A_{ij} - \frac{k_i k_j}{2m} \right).$$

Εκτελώντας τους υπολογισμούς βρίσκουμε ότι:

$$S_A = 0 - \frac{3^2}{16} = -\frac{9}{16}, \quad S_B = 2 - \frac{(2+3)^2}{16} = \frac{7}{16},$$

$$S_C = 2 - \frac{(3+2)^2}{16} = \frac{7}{16}, \quad S_D = 0 - \frac{3^2}{16} = -\frac{9}{16}.$$

Άρα

$$S = S_A + S_B + S_C + S_D = -\frac{1}{4},$$

και τελικά

$$Q = \frac{1}{2m} S = \frac{1}{16} \left(-\frac{1}{4} \right) = -\frac{1}{64} \approx -0,0156.$$

- (γ) - Η τιμή του modularity γίνεται σε σύγκριση με ένα τυχαίο γράφημα (τυχαία κατανομή ακμών).
Στην πρώτη διαμέριση παρατηρούμε ότι το modularity ήταν μεγαλύτερο από της δεύτερης διαμέρισης, επομένως η πρώτη διαμέριση του γραφήματος ήταν καλύτερη.
- Επίσης το modularity της πρώτης διαμέρισης ήταν θετικό, επομένως καλύτερο από το μοντέλο της τυχαία κατανομής ακμών, ενώ η δεύτερη διαμέριση ήταν αρνητική και επομένως χειρότερη από την τυχαία κατανομή ακμών.